Lab 1 : Data Preparation

Part 1 : Introduction to Data Science with Pandas

Exercise 1 : Exploring Vehicle Data with Pandas

The goal of the exercise is practice exploring and analyzing vehicle data using basic Pandas commands.

1. Load the CSV file (`cars.csv`) containing vehicle information
2. Show columns, types, and dimensions
3. Count unique values in the column **fuel_type**
4. Display the **average mileage** of all vehicles.
5. Show the **top 5 most frequent brands** in the dataset.
6. Display all vehicles whose **price is above the average price**.
7. Sort the dataset by **vehicle_age (oldest first)** and show the **top 5 oldest cars** with their **brand** and **price only**.

Exercise 2 :

The goal of the exercise is practice exploring, filtering, and summarizing real-world sales data using Pandas commands introduced in the lab.

1- After loading the Excel file (`sales_data.xlsx`), display the **last 5 rows** of the dataset.
2- Show the **unique categories** available in the column category and how many there are.
3- Find the **number of orders per country** and display the **top 3 countries** with the highest number of sales.
4- Calculate the **average order_value_EUR** for the category **"Games"** only.
5- Display all rows where the **profit margin** (difference between order_value_EUR and cost) is **greater than the average profit margin** of the dataset.
6- For each product category, calculate and display the **total profit generated**.
7- How many orders **were placed in the year 2020**? What was **the total revenue** generated during that year?
8- For each **sales_manager**, calculate their **average profit**. Then, display the managers whose **average profit is higher than the overall average profit in the dataset**.

Part 2: Data Cleaning

Exercise 3:

1. **Load** the dataset sales_data_1.xlsx into a DataFrame named df, and display the **first 5 rows**. Then identify data types and the number of missing values in each column.
2. Convert the column **date** to the correct datetime format. From this column, extract and create two new columns: **year** and **month**.
3. Check the column **cost** for any **non-numeric or invalid values**. Display them, then remove those rows from the dataset.
4. Replace missing values as follows: For **order_value_EUR** and **cost**, fill missing entries with their **mean value**. For **device_type**, fill missing entries with the **most frequent category (mode)**.
5. Verify that **order_value_EUR** and **cost** are stored as float. If not, convert them and confirm the change with df.dtypes.
6. Detect and remove **duplicate rows**. Report how many duplicates were removed.
7. Create a new column named **profit_margin** calculated as : profit_margin = order_value_EUR – cost; then display the first 5 rows showing this new column.
8. Display all rows where the **profit_margin** is **negative**. How many such cases exist? What might these represent in a business context?
9. **Standardize the text columns** — country, category, and city — to make them consistent and ready for analysis by removing extra spaces, converting to lowercase and replacing hyphens "-" with spaces. Display a few sample rows to verify that text values are standardized.
10. Create two **derived columns** :
    - margin_rate = (profit_margin / order_value_EUR)
    - profit_per_1000EUR = (profit_margin / 1000)

    Then compute and display the **top 5 categories** with the highest average margin_rate.
11. **Detect and correct business inconsistencies** :
    - Find all rows where order_value_EUR <= 0 or order_value_EUR < cost.
    - Count how many such transactions exist.
    - Remove them, and save the cleaned dataset as sales_data_1_cleaned.xlsx. (to save the file use df.to_excel("sales_data_1_cleaned.xlsx", index=False)
12. Manually add **three new rows** at the **top** of the DataFrame filled with NaN values. Then fill these missing values using **backward filling**. Display the top 10 rows before and after the operation to observe the change.
13. Use **MinMaxScaler** and **StandardScaler** from sklearn.preprocessing to scale the numeric columns order_value_EUR, cost, and profit_margin. Display the scaled values side-by-side.

Part 3: Grouping, Joins, and Aggregations

Exercise 4 :

The goal of the exercise is to practice organizing, summarizing, and merging vehicle data using Pandas aggregation and join functions.

1. **Load** the dataset `cars.csv` into a DataFrame named df. Group the dataset by **fuel_type** and calculate the total **km_driven** for each category.
2. Group the data by **transmission_type** and compute both the **mean price** and the **median engine** size using the agg() function.
3. Create a **pivot table** showing the **average price** of cars, organized by **brand** (rows) and **seller_type** (columns).
4. Count the number of vehicles per **brand** and sort the result in **descending order** to show the most frequent brands first.
5. Create a small DataFrame with **warranty** information for a few brands (عدد سنوات الضمان لكل علامة), then perform an **inner join** with the main dataset using the **brand** column.